Base 混合近似工具集技术说明

1. 技术方案原理

base_hybrid_approx 对应基础混合近似协议，用于在固定输入区间内计算 Sigmoid、Tanh、Erf、Exp、Inv 和 InvSqrt。它面向 秘密分享 场景：输入和输出均为二方分享，协议中间只允许打开被随机掩码保护后的值。

base 版本的核心假设是：输入已经落在该函数对应的拟合区间内。也就是说，本模块主要解决“如何在分享态下高效计算局部近似函数”，而不负责判断输入是否越界，也不负责区间外的饱和值选择。这样的分层可以让 base 协议保持结构简单、轮次较少，并作为 plus 协议的候选计算模块复用。

1.1 模块定位

base 混合近似协议服务于 Transformer 推理中的基础非线性函数：

函数	类型	典型上层用途	base 版本职责
Sigmoid	bounded	门控、SiLU 近似	固定区间内近似 S 形函数
Tanh	bounded	激活函数、近似表达	固定区间内近似奇函数
Erf	bounded	GeLU	近似误差函数的中间区间
Exp	unbounded	Softmax	稳定区间内近似指数函数
Inv	unbounded / 迭代型	Softmax 分母倒数	给出倒数初值并迭代修正
InvSqrt	unbounded / 迭代型	LayerNorm	给出倒平方根初值并迭代修正

从协议组合角度看，base approximation 输出仍然是加性分享，因此可以直接接入后续 SMP、3SMP 或上层算子，不需要公开中间近似值。

1.2 系统参与方与安全边界

base 协议仍然采用二方秘密分享加 TEE 辅助的结构：

角色	主要职责	可见信息
CSP0 / CSP1	持有输入分享，执行本地线性组合、网络交换和结果合成	自己的输入分享、输出分享、masked value、本地中间缓冲
TEE / Enclave	生成或保存与随机掩码相关的辅助材料	掩码 t 及其派生项、ECALL 输入、TEE 返回的辅助分享

角色	主要职责	可见信息
REE / App	调度协议流程、调用 ECALL、收发网络消息和计时	公开 masked value、本地缓冲、ECALL 参数和返回值

安全边界的核心是：REE 和网络可以看到 `d = x + t`，但不能看到原始 x；TEE 可以处理与 t 相关的辅助项，但不应获得两方完整输入分享。协议输出也保持为分享形式，不在在线计算阶段公开近似结果。

1.3 混合近似形式

核心思想是把非线性函数写成低阶多项式项和三角基函数的组合，将该类协议记为私有混合近似协议 Π_θ，其中 K_1 表示多项式最高阶，K_2 表示三角基数量：

```
f(x) ~= a₀ + a₁*x + a₂*x² + a₃*x³ + sum(Bₖ * sin(k*pi*x/L))
```

多项式项负责修正整体趋势，三角项负责提升拟合能力。相比只用高阶多项式，混合近似可以降低乘法深度；相比只用 sine 近似，它能减少振荡误差。base 版本假设输入落在拟合区间内，因此不额外执行宽范围分段选择。

其中参数 Π_θ 可以理解为一组固定函数参数，包括多项式系数、三角项系数、输入缩放方式和拟合区间。不同函数、不同区间、不同精度目标会对应不同参数。协议执行时双方不会学习明文 x，而是在分享域内按同一组公开或预置参数计算近似值分享。

1.4 函数类别

bounded 函数如 Sigmoid、Tanh、Erf 在固定区间内可以直接拟合。Sigmoid 可先转换为奇函数形式 `sigmoid(x) - 1/2`；Tanh 和 Erf 本身近似为奇函数，更适合用 sine 基展开。
Exp、Inv、InvSqrt 是 unbounded 或非奇函数，base 版本通常只在局部区间内拟合，并通过缩放、平移或 Newton 迭代改善精度。

这一区分很重要。bounded 函数的区间外值通常会趋向常数，因此 plus 版本可以在外层做饱和选择；base 版本只负责中间区间。unbounded 函数没有简单的两端常数饱和，若输入范围过宽，单一近似会同时面对陡峭区域和平缓区域，误差会快速上升。因此 Exp base 通常配合 Softmax clipping 使用，Inv/InvSqrt base 则通常作为 Newton 迭代初值生成器。

2. 逻辑推导

2.1 混合近似通用结构

base 协议的核心是先用随机掩码安全打开一个偏移点，再把函数近似拆成多项式项和三角项。多项式项负责低阶趋势，三角项负责局部形状修正。

设输入分享满足 $x = x_0 + x_1 \bmod L$。协议生成随机掩码 t，并让双方只打开 masked value：

```
d = x + t
```

由于 t 对外随机，公开 d 不直接泄露 x。后续所有近似项都围绕 $x = d - t$ 展开：与公开 d 相关的系数可以在 REE 本地计算；与隐藏 t 相关的项由 TEE 或预生成辅助分享提供。

2.1.1 掩码打开与多项式项

协议通过随机掩码打开 $d = x + t$。由于 t 随机，公开的 d 不泄露 x。多项式项可按 x=d-t 展开：

```
a₁*x = a₁*d - a₁*t
x²  = d² - 2*d*t + t²
x³  = d³ - 3*d²*t + 3*d*t² - t³
```

其中与 d 相关的部分可以本地计算，与 t,t^2,t^3 相关的部分由 TEE 辅助生成分享。

若保留到 K_1=3，多项式部分可写为 $poly(x) = a_0 + a_1*x + a_2*x^2 + a_3*x^3$。代入 x=d-t 后，公开值 d,d^2,d^3 只作为系数参与本地线性组合，不泄露原始 x。需要安全处理的项集中在 t,t^2,t^3 及其与公开系数的乘加，这正是 TEE 掩码材料和基础乘法协议提供的内容。

更具体地说，REE 可以根据公开 d 形成所有公开系数，例如 a_1*d、a_2*d^2、a_3*d^3，但不能直接计算 a_1*t、a_2*d*t、a_3*d^2*t 等包含隐藏掩码的项。实现中会把这些隐藏项拆成双方输出分享的一部分，使两方最终相加后等价于完整多项式值。

这种做法的优点是把非线性计算中的高成本部分集中到少量掩码辅助项上。公开 d 后，大量系数乘加都变成普通本地计算；协议只需要保护真正包含秘密输入信息的掩码相关项。

2.1.2 三角项

三角项利用差角公式：

```
sin(k*pi*x/L)
= sin(k*pi*(d-t)/L)
= sin(k*pi*d/L)cos(k*pi*t/L) - cos(k*pi*d/L)sin(k*pi*t/L)
```

d 公开后，sin(k*pi*d/L) 和 cos(k*pi*d/L) 本地可算；与 t 相关的 sin/cos 掩码项由协议生成分享。单个三角基在第 i 方的分享可以理解为 public_sin(d) * <cos(t)>_i - public_cos(d) * <sin(t)>_i。所有 $k=1..K_2$ 的三角基累加后，与多项式部分相加得到 base 近似输出分享。

这里的关键是差角公式把秘密输入上的三角函数 sin(k*x) 转换成公开点 d 上的三角函数和隐藏

掩码 t 上的三角函数组合。这样 REE 不需要直接计算 sin(k*x)，也不需要知道 x；它只用公开的 sin(k*d)、cos(k*d) 去线性组合 sin(k*t)、cos(k*t) 的分享。

三角项的作用不是替代多项式，而是补偿低阶多项式难以表达的局部弯曲。对 Tanh 和 Erf 这类近似奇函数，sine 基能较自然地表达中心区域的变化；对 Sigmoid，先减去 1/2 后得到近似奇函数，也能复用类似结构。

2.1.3 输出合成

base 协议最终输出可以理解为三部分相加：

```
<y> = <constant> + <poly(x)> + <trig(x)>
```

常数项可以由一方持有或按固定规则拆成分享；多项式项来自 x=d-t 的展开；三角项来自差角公式。只要每个组成部分都以加性分享形式给出，最终结果仍然是加性分享，可以继续进入 SMP、3SMP 或上层 GeLU/LayerNorm/Softmax 流程。

正确性是近似正确性：重构后的输出应接近目标明文函数，而不是等于精确实数函数。误差来自拟合参数、定点缩放、三角/多项式截断和后续安全乘法中的数值处理。

2.2 迭代型函数

Inv 和 InvSqrt 的 base 版本不能只依赖一次混合近似。混合近似先给出初值，随后 Newton 迭代在分享域内进一步收敛。

2.2.1 Inv 与 InvSqrt 的 Newton 迭代

Inv 和 InvSqrt 先用 hybrid approximation 得到初值，再通过 Newton 迭代提升精度：

```
Inv:      y <- y * (2 - x*y)
InvSqrt:  y <- y * (3 - x*y²) / 2
```

Inv 的迭代来自求解 1/x 的 Newton-Raphson 形式；InvSqrt 的迭代来自快速倒平方根形式。每次迭代只需要若干次安全乘法或三输入乘法，因此能够复用 SMP/3SMP。

对 Inv 来说，若当前估计为 y，则 x*y 越接近 1，y * (2 - x*y) 越接近真实倒数。对 InvSqrt 来说，若当前估计为 y，则 x*y² 越接近 1，y * (3 - x*y²) / 2 越接近真实倒平方根。

base hybrid approximation 在这里承担“初值生成”的角色。初值不需要完全精确，但必须落在 Newton 迭代可以收敛的区域内。若输入超出 base 拟合区间，初值可能偏离过大，后续迭代不仅收敛慢，还可能把误差放大。这也是 Inv+ 和 InvSqrt+ 需要分段初值的原因。

2.3 参数、定点表示与评估口径

base 协议中的近似参数不是在线学习得到的，而是在离线阶段针对固定函数和固定区间拟合好。

在线阶段只加载对应的 K_1/K_2、多项式系数、三角系数和缩放参数。这样做的好处是在线协议不需要执行复杂优化，只需要按固定公式组合分享；缺点是参数和输入区间绑定较强，区间外输入不能指望 base 版本自动保持精度。

所有实数值都会映射到模环 Z_L 上的定点整数。设缩放因子为 S，则明文实数 r 通常编码为 `round(r*S) mod L`。协议中的加法、减法和线性组合可以直接在模环中执行；乘法后需要按缩放因子回调，否则数值尺度会不断增大。三角函数和 Newton 迭代的辅助项也必须使用同一套缩放约定，否则即使代数公式正确，最终误差也会明显偏移。

因此评估 base_hybrid 时需要区分三类指标。第一是近似误差，包括平均误差和最大误差，用来判断参数是否适合当前区间。第二是数值稳定性，包括是否出现溢出、符号解释错误或缩放回调不一致。第三是协议开销，包括 ECALL 次数、网络通信量、SMP/3SMP 调用次数和端到端延迟。base 协议本身不做区间选择，所以若误差异常，首先应确认测试输入是否确实落在该函数的 base 拟合区间内。

3. 详细理论解释

Transformer 中需要大量复杂非线性函数，LUT 方法在输入范围变宽时表规模增长快，纯多项式或纯三角近似又容易在精度和开销之间失衡。base 版本的每个协议都使用同一类 private hybrid approximation，但每个函数的形状、区间和误差来源不同，因此需要分别理解。

3.1 共同安全与误差模型

base 协议的安全性可以从三个方面理解。

第一，公开的是 d=x+t，不是 x。只要 t 新鲜且不泄露，单次公开 d 不会直接暴露输入。第二，与 t 相关的多项式和三角辅助项只以分享形式参与输出合成，REE 不应看到完整的 t、t^2、$\sin(t)$ 或 $\cos(t)$。第三，协议输出仍是分享，上层算子只能继续在分享域中组合，不能在中间打开近似值。

误差来源主要包括：

来源	影响
拟合区间	输入越接近区间边界，局部误差可能越大
多项式阶数 K_1	阶数越高表达能力越强，但乘法和补偿复杂度更高
三角基数量 K_2	基数量越多局部拟合更强，但辅助项和本地组合开销更高
定点缩放	缩放不足会带来量化误差，缩放过大可能增加溢出风险
Newton 初值	Inv/InvSqrt 的收敛速度和最终精度依赖初值质量

因此 base 文档中的“正确性”应理解为：在目标拟合区间和固定参数下，重构输出与明文 reference 的误差低于设定阈值。

3.2 Bounded 函数 base 协议

Sigmoid、Tanh 和 Erf 都是有界函数。base 版本只负责固定拟合区间内的近似，不处理区间外饱和选择；这三类函数的主要差异在于是否天然具备奇函数结构，以及误差会如何影响上层算子。

3.2.1 Sigmoid base 协议

Sigmoid base 在固定区间内近似 `sigmoid(x) = 1 / (1 + exp(-x))`。Sigmoid 不是奇函数，但可以写成 `1/2 + (sigmoid(x) - 1/2)`，其中 `sigmoid(x)-1/2` 是奇函数形式，适合用 sine 基拟合。协议先用随机掩码打开 d=x+t，再用差角公式计算每个 `sin(k*x)` 的分享，同时用低阶多项式修正中心区间的趋势误差。Sigmoid base 的正确性来自混合近似公式：多项式项和三角项相加后得到 `sigmoid(x)` 的近似分享。安全性来自 d=x+t 的掩码保护，CSP 只看到公开的 d 和自己的分享，TEE 只看到与 t 相关的被保护材料。

Sigmoid base 只适用于拟合区间内。区间外 Sigmoid 快速趋向 0 或 1，base 版本不负责饱和处理；这部分由 Sigmoid+ 完成。

Sigmoid 的误差通常在区间两侧更敏感。中心区域斜率较大，要求近似函数准确表达 S 形变化；远离中心后函数趋于平坦，base 区间若设置过宽，低阶多项式和有限三角基需要同时拟合陡峭中心和两端平坦区域，误差会变大。因此 Sigmoid base 更适合作为中间区间近似，区间外交给 Sigmoid+ 的 gate 和饱和值处理。

3.2.2 Tanh base 协议

Tanh base 在固定区间内近似 `tanh(x) = (exp(x) - exp(-x)) / (exp(x) + exp(-x))`。Tanh 本身是奇函数，天然适合 sine 基展开。协议仍然打开 d=x+t，本地计算 `sin(k*d)` 和 `cos(k*d)`，通过 TEE/掩码材料得到 `sin(k*t)` 和 `cos(k*t)` 的分享，再按差角关系 `sin(k*(d-t)) = sin(k*d)cos(k*t) - cos(k*d)sin(k*t)` 组合。Tanh base 的多项式项用于修正 sine 近似在边界和中心区域的误差。正确性来自每个三角基都按差角公式还原为 `sin(k*x)` 的分享，所有基函数加权后得到 `tanh(x)` 的近似。安全性和 Sigmoid base 一样：打开的是 x+t 而不是 x，三角掩码项只以分享形式存在。

Tanh 的优势是函数关于原点反对称，因此常数项压力较小，sine 基可以直接表达主要形状。它的挑战在于边界处逐渐饱和，若 base 区间覆盖过大，有限基函数可能出现振荡或边界误差。实际使用时，Tanh base 的区间通常应覆盖模型激活值的密集区域，而不是试图覆盖所有极端输入。

3.2.3 Erf base 协议

Erf base 近似误差函数 `erf(x) = 2/sqrt(pi) * integral_0^x exp(-t²) dt`。Erf 也是奇函数，并且在 `GeLU(x) = 0.5*x*(1 + erf(x/sqrt(2)))` 中直接使用，因此 Erf base 的精度会直接影响 GeLU。协议结构与 Tanh base 类似：用 d=x+t 打开公开点，用 sine 基表达局部形状，用低阶多项式降低残差。Erf 在中心区域变化明显、两端逐渐饱和，所以 base 版本主要用于中间拟合区间。正确性由混合近似公式保证，安全性由掩码打开和分享化三角项保证。

Erf base 对上层 GeLU 尤其重要。GeLU 中会把 `erf(x/sqrt(2))` 与 x 相乘，因此 Erf 的近似误差会被输入幅值放大。若 Erf base 在尾部误差较大，GeLU 输出会出现系统偏差。为避免这一点，宽范围 GeLU 应优先使用 Erf+，让区间外输入通过饱和逻辑处理，base 只负责中心区间。

3.3 Unbounded 与迭代型 base 协议

Exp、Inv 和 InvSqrt 对输入范围更敏感。Exp 增长很快；Inv 和 InvSqrt 在小输入附近变化剧烈。因此 base 版本通常只承担局部区间近似，宽范围支持交给 plus 协议。

3.3.1 Exp base 协议

Exp base 近似 `exp(x)`。Exp 增长快，直接在宽区间内拟合会非常困难。因此 base 版本通常只用于局部区间，例如 Softmax clipping 后的稳定区间。协议仍使用与 Sigmoid/Tanh/Erf 相同的 hybrid approximation 结构，通过多项式项和三角项组合得到输出分享，区别主要在拟合区间和参数取值。

Exp base 的正确性是近似正确性：输出分享相加后不是精确 `exp(x)`，而是在目标区间内接近 `exp(x)`。安全性仍与其他 base 协议一致：公开 d=x+t 不泄露 x，所有与 t 相关的项由 TEE/协议以分享形式提供。Exp base 不解决宽范围指数增长问题，Exp+ 会在其外层加入 clipping 和分段。

Exp 的误差会直接影响 Softmax 的分子，也会通过行求和影响分母。因此 Exp base 的目标不是“全范围指数函数”，而是“在安全裁剪后的稳定区间内提供足够准确的指数近似”。如果输入没有经过 clipping 或区间控制，指数增长会使局部拟合快速失效。

3.3.2 Inv base 协议

Inv base 近似 `inv(x) = 1 / x`。倒数函数在靠近 0 时变化剧烈，因此 base 版本只适合远离 0 的固定区间。协议先用 hybrid approximation 得到初值 y_0，再执行 Newton 迭代 `y <- y * (2 - x*y)`。如果 y 已经接近 1/x，则 x*y 接近 1，2-x*y 是修正因子。每轮迭代需要安全乘法：先计算 x*y，再计算 y*(2-x*y)。Inv base 的正确性由“近似初值 + Newton 收敛”共同保证。安全性由 hybrid approximation 和 SMP 的组合安全保证；迭代过程中所有中间值仍为分享。

Inv base 常用于 Softmax 的分母倒数。Softmax 行和通常为正，但数值范围会随行长度、clipping 区间和 Exp 误差变化。若行和落在 base 拟合区间外，倒数初值可能不够好，Newton 迭代轮数增加也未必能完全修正。因此宽范围 Softmax 通常需要 Inv+ 的分段初值。

3.3.3 InvSqrt base 协议

InvSqrt base 近似 `invsqrt(x) = 1 / sqrt(x)`。该协议主要服务于 LayerNorm 中的 `1/sqrt(var+eps)`。与 Inv 类似，InvSqrt 在小输入区域变化快，base 版本只适合固定区间。协议先通过 hybrid approximation 得到初值 y_0，再执行 Newton 迭代 `y <- y * (3 - x*y²) / 2`。该迭代需要计算 y^2、$x*y^2$ 和最终乘法，因而会调用 SMP/3SMP。InvSqrt base 的

正确性依赖初值质量和迭代轮数；若输入超出拟合区间，初值误差可能较大，后续 LayerNorm 输出会偏移。安全性由 base hybrid approximation、SMP/3SMP 和分享域线性操作组合得到，协议过程中不会公开 x、y 或 var 明文。

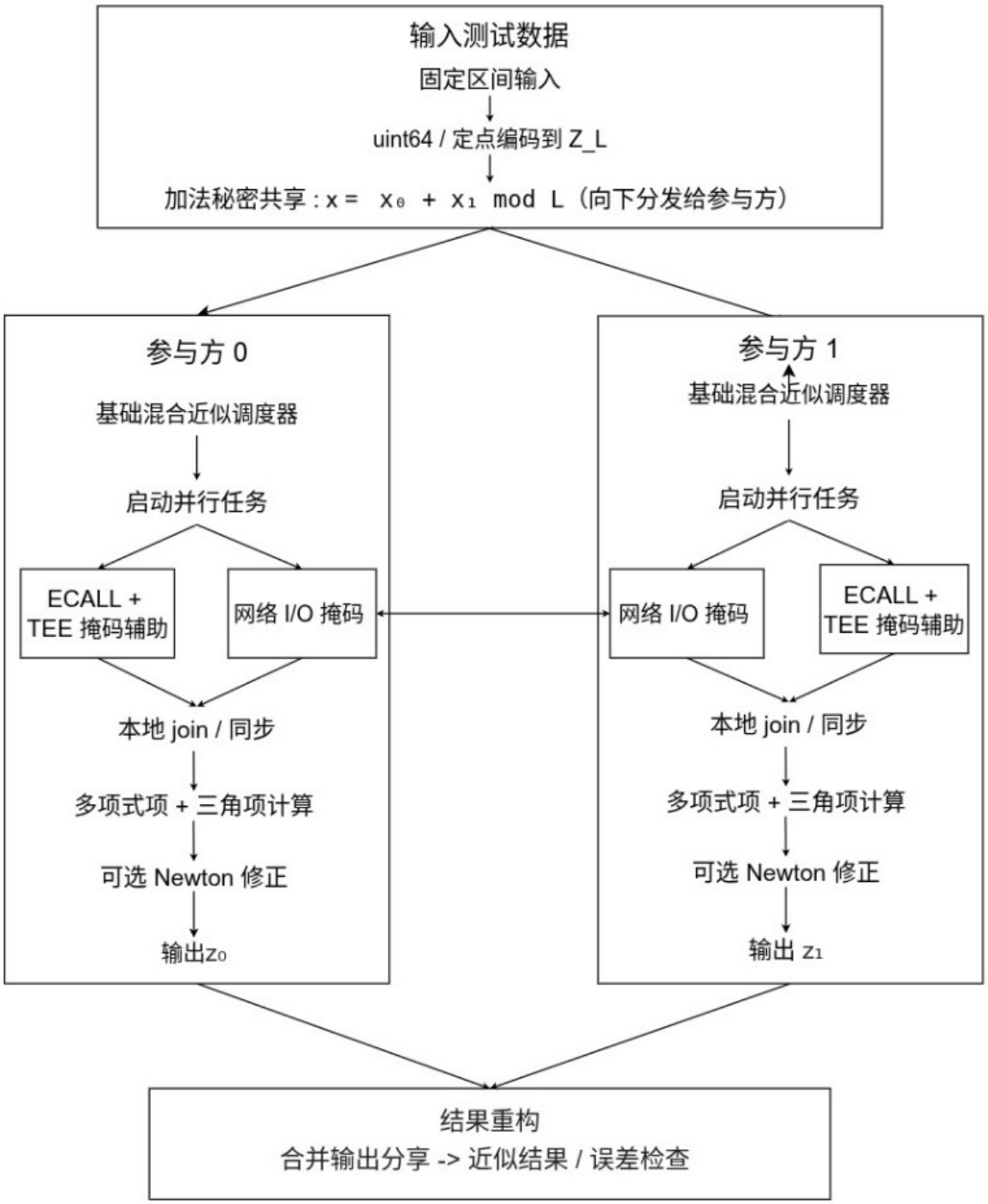
InvSqrt 的误差会系统性影响 LayerNorm 一整行输出，因为同一个 `inv_std` 会乘到该行所有归一化元素上。对小方差输入，`1/sqrt(var+eps)` 变化非常敏感；对大方差输入，函数变化较平缓但尺度差异大。因此 base InvSqrt 更适合作为某一局部区间的初值生成器，宽范围场景应使用 InvSqrt+ 的 steep/flat 分段。

3.4 与 plus 协议的关系

base 协议和 plus 协议不是替代关系，而是内外层关系。base 负责固定区间内的候选值计算；plus 负责判断输入属于哪个区间、选择饱和值或分段候选，并在必要时执行 Newton refinement。

这种分层带来两个好处。第一，base 协议可以保持简单，便于独立测试每个函数的局部近似误差。第二，plus 协议可以复用 base 作为候选计算模块，只在外层增加比较和选择逻辑。调参时也可以分开看：先确认 base 在目标区间内的平均误差和最大误差，再确认 plus 的区间划分是否覆盖真实输入分布。

4. 算法框架图


输入测试数据
固定区间输入
uint64 / 定点编码到 Z_L
加法秘密共享 : x = x₀ + x₁ mod L（向下分发给参与方）
参与方 0
基础混合近似调度器
启动并行任务
ECALL + TEE 掩码辅助
网络 I/O 掩码
本地 join / 同步
多项式项 + 三角项计算
可选 Newton 修正
输出z₀
参与方 1
基础混合近似调度器
启动并行任务
网络 I/O 掩码
ECALL + TEE 掩码辅助
本地 join / 同步
多项式项 + 三角项计算
可选 Newton 修正
输出 z₁
结果重构
合并输出分享 -> 近似结果 / 误差检查

5. 数据流程图


1. 明文输入或测试向量
输入固定区间的明文 / 测试向量
2. 编码
将输入定点编码到 Z_L
3. 秘密分享
选择随机 x0
令 x1 = x - x0 mod L
4A. CSP0 本地设置
- 选择函数类型:
Sigmoid/Tanh/Erf/Exp/Inv/Rsqrt
- 加载 K1/K2 系数
- 准备随机掩码 t
- 准备 ECALL / 网络缓冲区
4B. CSP1 本地设置
- 选择函数类型:
Sigmoid/Tanh/Erf/Exp/Inv/Rsqrt
- 加载 K1/K2 系数
- 准备随机掩码 t
- 准备 ECALL / 网络缓冲区
5A. CSP0 并行在线阶段
网络路径
- 交换掩码分享
- 重构公开 d = x + t
TEE / 本地辅助路径
- 生成辅助分享t, t², t³
- 生成辅助分享sin t/cos t
- 根据公开d计算本地基函数
本地 join：等待两条路径完成
5B. CSP1 并行在线阶段
网络路径
- 交换掩码分享
- 重构公开 d = x + t
TEE / 本地辅助路径
- 生成辅助分享t, t², t³
- 生成辅助分享sin t/cos t
- 根据公开d计算本地基函数
本地 join：等待两条路径完成
6A. Base 近似合成
- Sigmoid/Tanh/Erf/Exp:
多项式项 + 三角项生成 base 输出分享
- Inv / InvSqrt:
多项式项 + 三角项生成 hybrid 初值 + SMP/3SMP Newton 更新生成输出分享
6B. Base 近似合成
- Sigmoid/Tanh/Erf/Exp:
多项式项 + 三角项生成 base 输出分享
- Inv / InvSqrt:
多项式项 + 三角项生成 hybrid 初值 + SMP/3SMP Newton 更新生成输出分享
7. 验证 / 统计
计算z = z0 + z1 mod L
将结果与明文数学函数进行比较

6. 运行方式

在 `party0` 和 `party1` 目录分别执行：

```
./1.sh
```

测试宏由 `App/define/define.h` 控制。输出会显示不同 K_1,K_2 配置下的耗时、平均误差、最大误差和通信统计。